compte rendu

TP 2 : Les sockets en mode connecté TCP : E/R de CC & objets

Activité 2-1 : Échange de CC entre Client/Serveur via les Sockets TCP

objective :

- Dans cette activité, le client envoie les éléments d'une opération sous forme de chaînes de caractères (premier opérande, opérateur, second opérande) au serveur, puis le serveur valide, calcule et renvoie le résultat.

- ***programme partie client***

```java
package Client;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.Scanner;
public class client {
        public static void main(String[] args) {
            String adresseServeur = "127.0.0.1";
            int port = 1234;

            try (Socket socket = new Socket(adresseServeur, port);
                 Scanner scanner = new Scanner(System.in)) {

                System.out.println("Connecté au serveur !");

                // Flux pour l'échange de chaînes de caractères
                BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);

                // Saisie des données
                System.out.print("Entrez le premier opérande : ");
                String op1 = scanner.nextLine();

                System.out.print("Entrez l'opérateur (+, -, *, /) : ");
                String op = scanner.nextLine();

                System.out.print("Entrez le deuxième opérande : ");
                String op2 = scanner.nextLine();

                // Envoi des données au serveur
                out.println(op1);
```

```
                out.println(op);
                out.println(op2);

                // Réception et affichage du résultat
                String reponse = in.readLine();
                System.out.println("Réponse du serveur : " + reponse);

            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }

```

- ***programme partie serveur***

```
package Serveur;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class serveur {

    public static void main(String[] args) {
        int port = 1234;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Serveur en attente de connexion sur le port " + port + "...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connecté !");

            // Flux de lecture et d'écriture de texte (chaînes de caractères)
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);

            // Lecture des 3 éléments envoyés par le client
            String op1Str = in.readLine();
            String operateur = in.readLine();
            String op2Str = in.readLine();

            // Validation de la syntaxe et calcul
            try {
                double op1 = Double.parseDouble(op1Str);
                double op2 = Double.parseDouble(op2Str);
                double resultat = 0;
                boolean erreur = false;
                switch (operateur) {
                    case "+":
                        resultat = op1 + op2;
                        break;
                    case "-":
                        resultat = op1 - op2;
                        break;
                    case "*":
                        resultat = op1 * op2;
                        break;
                    case "/":
                        if (op2 != 0) {
                            resultat = op1 / op2;
                        } else {
                            out.println("Erreur : Division par zéro !");
                            erreur = true;
                        }
                        break;
                    default:
                        out.println("Erreur : Opérateur invalide !");
                        erreur = true;
                        break;
                }

                if (!erreur) {
                    out.println("Résultat : " + resultat);
                }
            } catch (NumberFormatException e) {
                out.println("Erreur : Syntaxe des opérandes incorrecte !");
```

```
63              }
64
65              socket.close();
66              System.out.println("Connexion fermée.");
67          } catch (Exception e) {
68              e.printStackTrace();
69          }
70      }
71  }
```

- ***resultat du code***

```
Serveur en attente de connexion sur le port 1234...
Client connecté !
Connexion fermée.
```

```
Connecté au serveur !
Entrez le premier opérande : 7
Entrez l'opérateur (+, -, *, /) : +
Entrez le deuxième opérande : 3
Réponse du serveur : Résultat : 10.0
```

compte rendu

TP 2 : Les sockets en mode connecté TCP : E/R de CC & objets

Activité 2-2 : Échange des objets entre Client/Serveur via les Sockets TCP

objective :

- *Dans cette activité, les éléments de l'opération sont encapsulés dans une classe Operation qui implémente Serializable*

- ***package serveur ,classe :serveurObject***

```java
package Serveur;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.ServerSocket;
import java.net.Socket;
import model.Operation;
public class ServeurObject {  new*
        public static void main(String[] args) {  new*
            int port = 1234;
            try (ServerSocket serverSocket = new ServerSocket(port)) {
                System.out.println("Serveur (Objets) en attente sur le port " + port + "...");
                Socket socket = serverSocket.accept();

                // Flux d'objets
                ObjectInputStream in = new ObjectInputStream(socket.getInputStream());
                ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream());

                // Réception de l'objet Operation[cite: 4]
                Operation op = (Operation) in.readObject();
                // Effectuer le calcul[cite: 4]
                double res = 0;
                boolean valide = true;

                switch (op.getOperateur()) {
                    case '+': res = op.getOperande1() + op.getOperande2(); break;
                    case '-': res = op.getOperande1() - op.getOperande2(); break;
                    case '*': res = op.getOperande1() * op.getOperande2(); break;
                    case '/':
                        if (op.getOperande2() != 0) {
                            res = op.getOperande1() / op.getOperande2();
```

```
                    } else {
                        op.setErreur("Erreur : Division par zéro !");
                        valide = false;
                    }
                    break;
                default:
                    op.setErreur("Erreur : Opérateur inconnu !");
                    valide = false;
                    break;
            }

            if (valide) {
                op.setResultat(res);
            }

            // Renvoi de l'objet modifié au client[cite: 4]
            out.writeObject(op);

            socket.close();
            System.out.println("Traitement terminé.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

- ***package model ,classe : operation***

```
package model;
import java.io.Serializable;
public class Operation implements Serializable {  no usages  new *  2 related problems
    // La classe doit implémenter Serializable pour être transmise via Sockets[cite: 3, 4]
    private static final long serialVersionUID = 1L;  no usages
    private double operande1;  2 usages
    private double operande2;  2 usages
    private char operateur;  2 usages
    private double resultat;  2 usages
    private String erreur;  2 usages
    public Operation(double operande1, double operande2, char operateur) {  no usages  new *  2 related problems
        this.operande1 = operande1;
        this.operande2 = operande2;
        this.operateur = operateur;
    }
    public double getOperande1() { return operande1; }  no usages  new *
    public double getOperande2() { return operande2; }  no usages  new *
    public char getOperateur() { return operateur; }  no usages  new *
    public double getResultat() { return resultat; }  no usages  new *  1 related problem
    public void setResultat(double resultat) { this.resultat = resultat; }  no usages  new *
    public String getErreur() { return erreur; }  no usages  new *  2 related problems
    public void setErreur(String erreur) { this.erreur = erreur; }  no usages  new *
}
```

- ***package client ,classe : clientObject***

```java
package Client;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.Socket;
import java.util.Scanner;
public class ClientObject { new*
        public static void main(String[] args) {  new*
            String adresse = "127.0.0.1";
            int port = 1234;

            try (Socket socket = new Socket(adresse, port);
                 Scanner scanner = new Scanner(System.in)) {

                // Flux d'objets (Attention : créer ObjectOutputStream en premier)
                ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream());
                ObjectInputStream in = new ObjectInputStream(socket.getInputStream());

                // Saisie
                System.out.print("Premier opérande : ");
                double op1 = scanner.nextDouble();

                System.out.print("Opérateur (+, -, *, /) : ");
                char op = scanner.next().charAt(0);

                System.out.print("Deuxième opérande : ");
                double op2 = scanner.nextDouble();

                // Création et envoi de l'objet[cite: 4]
                Operation operation = new Operation(op1, op2, op);
                out.writeObject(operation);

                // Réception de l'objet retourné par le serveur[cite: 4]
                Operation opResultat = (Operation) in.readObject();

                if (opResultat.getErreur() != null) {
                    System.out.println(opResultat.getErreur());
                } else {
                    System.out.println("Résultat calculé par le serveur : " + opResultat.getResultat()); //[cite: 4]
                }

            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }

```

- ***resultat du code***

```
Serveur (Objets) en attente sur le port 1234...
Traitement terminé.

Process finished with exit code 0
```

```
Premier opérande : 2
Opérateur (+, -, *, /) : *
Deuxième opérande : 6
Résultat calculé par le serveur : 12.0
```